

Project Report

Object Detection on a COCO Subset Using YOLOv5

1. Overview

This project implements an end-to-end object detection pipeline built on YOLOv5, trained and evaluated on a curated subset of the COCO 2017 dataset. The pipeline covers five stages: data preprocessing, model architecture selection, training and evaluation, real-time inference, and model deployment/optimization, followed by a robustness analysis.

Three YOLOv5 variants YOLOv5s (small), YOLOv5m (medium), and YOLOv5l (large) were fine-tuned and compared against each other on accuracy, inference speed, and robustness to visual perturbations.

2. Data Preprocessing

Source	Official COCO 2017 annotations accessed via the pycocotools COCO API.
Sampling strategy	Stratified sampling: up to 50 images per category across all 80 COCO categories, to ensure balanced class representation. This produced 4,000 unique images (some images contain multiple categories, so the unique count is naturally below the theoretical 4,000-image ceiling).
Category coverage	All 80 categories had non-zero representation in the sample. The least-represented category (hair drier) had 53 images; the most-represented (person) had 2,110 images reflecting COCO's natural class imbalance.
Preprocessing steps	• Images were letterbox-resized to 640×640, preserving aspect ratio with padding. • Bounding box annotations were remapped through the same resize/padding transform and converted to normalized YOLO format.
Dataset split	• 80% train, 10% validation, 10% test, applied to 4,000 preprocessed image-label pairs: ◦ Train: 3,200 images ◦ Validation: 400 images ◦ Test: 400 images

Output	A YOLOv5-compatible directory structure (images/, labels/ per split) plus a coco_subset.yaml config file listing paths, class count (80), and class names.
---------------	--

3. Model Architecture

Three pretrained YOLOv5 variants (pretrained on full COCO) were loaded via torch.hub and fine-tuned on the subset:

Variant	Parameters	GFLOPs	Layers
YOLOv5s	7.23M	16.4	214
YOLOv5m	21.17M	48.9	291
YOLOv5l	46.53M	109.0	368

All three share the same YOLOv5 architecture family: a Conv + C3 blocks, a Spatial Pyramid Pooling; Fast layer, a PANet-style neck for multi-scale feature fusion, and a multi-scale detection head predicting boxes at three resolutions. The variants differ only in width/depth scaling, trading capacity for speed.

4. Training Configuration

All three variants were fine-tuned under an identical training configuration for a fair comparison:

- **Epochs:** 30
- **Batch size:** 16
- **Image size:** 640×640
- **Optimizer:** AdamW (lr = 0.01)
- **LR schedule:** Cosine annealing
- **Loss:** CIoU (box) + objectness + classification loss, YOLOv5's default combination
- **Augmentation:** Mosaic, HSV color jitter, horizontal flip, translation/scale jitter, and light Albumentations-based blur/gray/CLAHE augmentations
- **Hardware:** Single NVIDIA Tesla T4 GPU (Google Colab)

Training time per variant:

Variant	Training time
YOLOv5s	0.735 hours
YOLOv5m	0.985 hours
YOLOv5l	1.843 hours

Larger variants took proportionally longer per epoch due to higher GPU memory usage and compute per forward/backward pass.

5. Evaluation Results

Evaluation was performed two ways:

(1) YOLOv5's built-in validation during training.

(2) An independent COCO-style evaluation using pycocotools.COCOeval against a custom-generated ground-truth JSON for the held-out validation split.

5.1 Accuracy comparison (COCOeval, held-out validation set)

Variant	Params (M)	mAP@0.5:0.95	AP50	AP75
YOLOv5s	7.23	0.1645	0.2883	0.1658
YOLOv5m	21.17	0.1785	0.2964	0.1907
YOLOv5l	46.53	0.1830	0.3017	0.1977

YOLOv5l achieved the best accuracy across all three metrics, consistent with its larger capacity. The improvement over YOLOv5s is meaningful; expected given all three were trained under identical, relatively short (30-epoch) schedules on a modest 3,200-image training set rather than full COCO.

5.2 Inference speed comparison

Benchmarked on 30 held-out test images (Tesla T4 GPU, batch size 1, after 5 warmup runs):

Variant	Avg. inference time	FPS
YOLOv5s	21.07 ms	47.46
YOLOv5m	23.54 ms	42.48
YOLOv5l	38.39 ms	26.05

YOLOv5s is roughly $1.8\times$ faster than YOLOv5l, while YOLOv5l delivers only a modest accuracy gain a classic accuracy/speed tradeoff. For real-time or edge deployment where latency matters, YOLOv5s or YOLOv5m may be preferable despite slightly lower accuracy.

5.3 Robustness testing

YOLOv5l was tested on images after applying three artificial changes (noise, blur, and low light) and compared to original images.

Condition	Avg. detections/image	Avg. confidence	% of baseline detections
Baseline (no perturbation)	4.82	0.561	100.0%
Low light (0.4× brightness)	4.18	0.560	86.7%
Bright light (1.8× brightness)	4.28	0.572	88.8%
Occlusion (25% random patch)	4.82	0.546	100.0%

Scale variation (50% smaller)	3.28	0.592	68.0%
-------------------------------	------	-------	-------

Key findings:

- The model was most robust to occlusion detection count was unaffected, though average confidence dropped slightly (0.561 $\rightarrow$ 0.546).
- Lighting changes (both darker and brighter) caused a moderate ~11–13% drop in detection count.
- Scale variation caused the largest degradation a 32% drop in detections indicating the model is most sensitive to object scale changes, consistent with the earlier finding that small-object AP was lower than large-object AP.

6. Deployment and Optimization

The best variant (YOLOv5l, selected by highest mAP@0.5:0.95) was prepared for edge deployment:

- **Model export:** Converted the model to ONNX and TorchScript formats so it can run on different devices and platforms.
- **Dynamic quantization:** Reduced the model's precision to INT8 to improve CPU performance on low-power devices. However, for YOLOv5, this did not reduce the model size because the model mainly uses convolutional layers.
- **Magnitude pruning:** Removed 30% of less important convolutional weights to reduce the model's complexity and saved it as a separate model file.

7. Real-Time Detection System

A working inference pipeline was built using OpenCV + PyTorch:

- **Image inference** (detect_image): loads an image, runs the detector, renders bounding boxes, and saves the annotated result.
- **Video inference** (detect_video): processes a video file frame-by-frame, timing each inference call and reporting average FPS, with the annotated video written to disk.

Both functions load the best fine-tuned model (YOLOv5l in this run) with confidence threshold 0.35 and NMS IoU threshold 0.45.

8. Summary and Recommendations

- **Best accuracy:** YOLOv5l (mAP@0.5:0.95 = 0.183)
- **Best speed:** YOLOv5s (47.46 FPS, ~2× faster than YOLOv5l)
- **Best balance:** YOLOv5m offers a middle ground (mAP 0.1785, 42.48 FPS) if neither extreme is required.

- **Weakest point across all variants:** small-object detection and robustness to scale variation both point to the same underlying limitation, likely a combination of the limited training set size (3,200 images) and relatively short 30-epoch training schedule.
- **Recommendation for improvement:** increasing training epochs and/or dataset size would likely improve mAP further, since all three variants' loss curves were still decreasing at epoch 30 without clear signs of convergence or overfitting.